



ITHILIEN

The Plant Care Tracking App

Design Documentation

SENG3080

Advanced Web Frameworks

Conestoga College

04/02/2026

Josh Horsley, Josh Rice, Kalina Cathcart, Mehr Keshishian,
Christian Tan

TABLE OF CONTENTS

Introduction	1
Architectural Overview	3
Full list of Architecture components	3
Interactions	3
Planned Features	4
Web UI Mockup	5
Landing Page	5
Login / Register	6
My Garden (Home)	7
Add Plant modal	8
Plant Profile	9
Calendar View	10
User Settings	11
APIs and Interfaces	12
Data Flow	12
External API: Perenual.com	13
Species List	14
Plant Details	15
Internal API	16
Endpoints:	16
Error Handling	17
Deployment and Release Strategy	18
Local Dev Environment	18
Environment Variables	19
Image Storage Adapter	19
Deployment to Azure App Services	19

INTRODUCTION

Ithilien is a plant care tracking web application designed to help users monitor and manage their houseplants' needs. Many plant owners struggle to remember watering schedules, light requirements, and seasonal care adjustments, often resulting in neglected or overwatered plants. Ithilien solves these issues by providing a centralized dashboard for tracking plant care routines and sending out timely reminders.

Target Audience: Houseplant owners ranging from beginners to hobbyists who want a simple organized way to keep their plants happy and healthy.

Stakeholders: End users, the development team and the course instructor.

What's in a name: The name "Ithilien" is inspired by the garden region of Gondor in Lord of the Rings, reflecting the app's focus on nurturing plant life.

ARCHITECTURAL OVERVIEW

Ithilien is a cloud-hosted web application made in React + ExpressJS. It will leverage Azure services for all deployment and storage concerns, and Perenual API, a service available to accelerate horticulture solutions.

FULL LIST OF ARCHITECTURE COMPONENTS

Libraries and Frameworks	
React	Frontend framework
NodeJS / Express	Backend framework
Bootstrap	UI Component Library
Prisma	ORM/query builder for MySQL interactions
Better-auth	JWT/session handling library
Cloud Services	
Azure App Services	Managed web deployment service
Azure Blob Storage	Image and file storage
Azure MySQL	Relational DB
Additional Services	
Perenual API	Domain-specific API with data on plant species, care guides, etc. Found at: https://perenual.com/

INTERACTIONS

Ithilien will be a dynamic web application, with Azure Web Apps providing a containerized endpoint for both the frontend and backend. Both are deployed within a single Azure Web App for simplicity.

All operations involving MySQL, blob storage, or the Perenual API will occur via API endpoints defined in our ExpressJS backend, allowing our server to manage authentication and scope for all interactions.

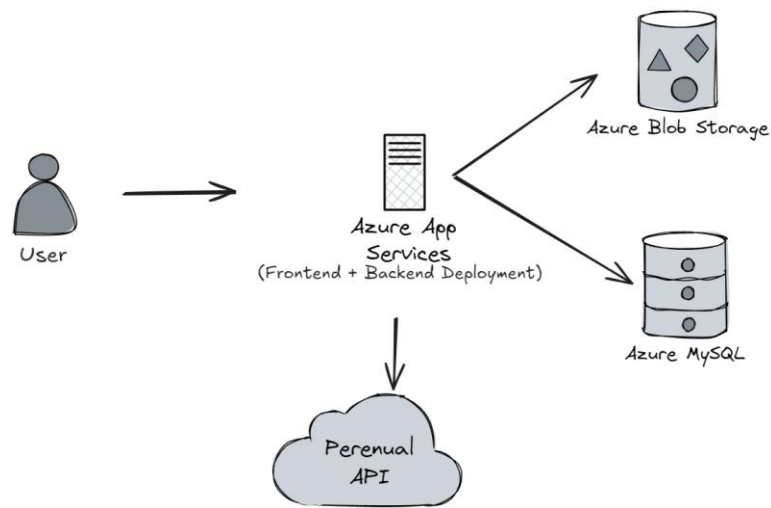


Fig 1 – High level cloud architecture for Ithilien.

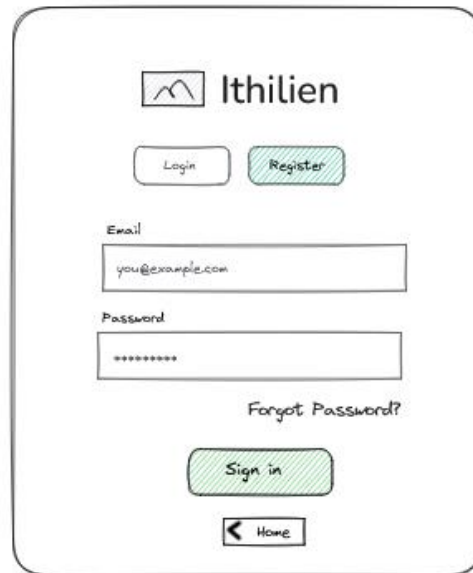
PLANNED FEATURES

Ithilien aims to be a robust plant care tracking application with several important features:

1. **Authentication:** Users can register, log in, and log out of the application
2. **User Profile Management:** Logged-in users can update their email, password, and time zone preferences.
3. **My Garden (Home View):** Users can view their plant collection with a summary of each plant's care status, sorted by care urgency (plants needing attention soonest appear first). A dropdown menu on each plant provides quick actions (mark as watered, edit, delete).
4. **Plant Profiles:** Users can select a plant species, assign a custom nickname, upload user photos and view care information pulled from the Perennial API. Users can also edit or delete plants from this view.
5. **Care Tracking:** Users can mark plants as watered and view a care history log showing past actions, for each plant.
6. **Calendar View:** Displays an up-to-date watering schedule showing upcoming and past care tasks
7. **Reminder System:** The app sends notifications to remind users when their plants need watering or other care.

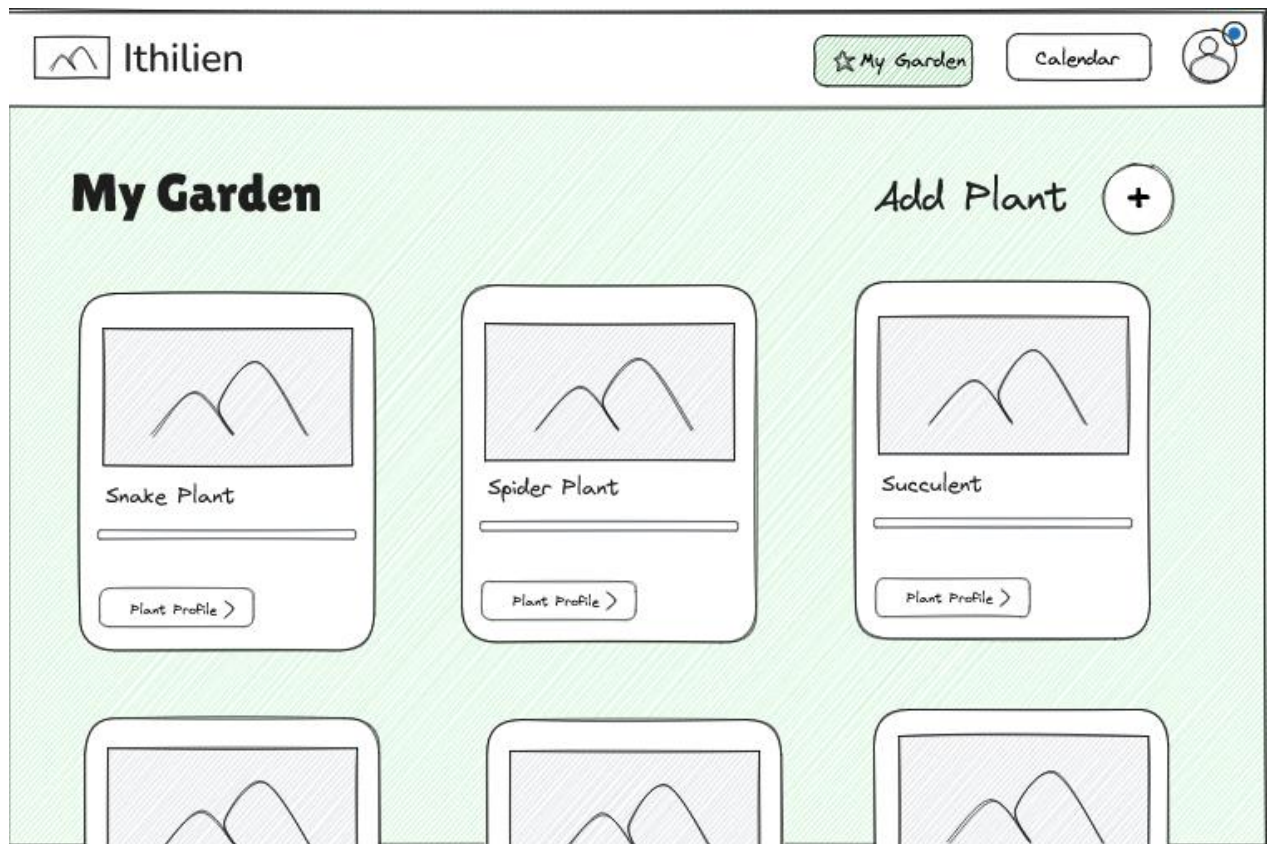


The Landing Page serves as the public-facing entry point for Ithilien. It features a hero section with the app's tagline, a brief description, and a call-to-action button prompting users to register. Below the hero, three feature cards highlight the app's core functionality: plant tracking, smart reminders, and the calendar view. The navigation bar includes links to login and register.



The form is a rounded rectangle with a light gray background. At the top center is a logo consisting of a square icon with a stylized mountain range and the text "Ithilien" to its right. Below the logo are two buttons: "Login" (white with a thin gray border) and "Register" (green with a diagonal line pattern). Under these buttons are two input fields. The first is labeled "Email" and contains the text "you@example.com". The second is labeled "Password" and contains seven asterisks "*****". To the right of the password field is a link that says "Forgot Password?". Below the input fields is a green button with a diagonal line pattern labeled "Sign in". At the very bottom is a small white button with a left-pointing arrow and the text "Home".

The Login/Register page allows users to toggle between login and registration modes using a tabbed interface. Existing users can sign in by entering their email and password. New users can register by providing an email, password, and password confirmation. A "Forgot password" link is available for account recovery, and a link returns users to the landing page.



The My Garden page serves as the main dashboard after logging in. Users can view all their plants at a glance, displayed as cards sorted by care urgency (plants needing attention soonest appear first). Each card shows the plant's photo, nickname, species, and current care status. A dropdown menu on each card provides quick actions: Mark as Watered, Edit Plant, and Delete Plant. The "Add Plant" button in the top corner allows users to add new plants to their collection. The header includes navigation links to the Calendar and Account Settings pages.

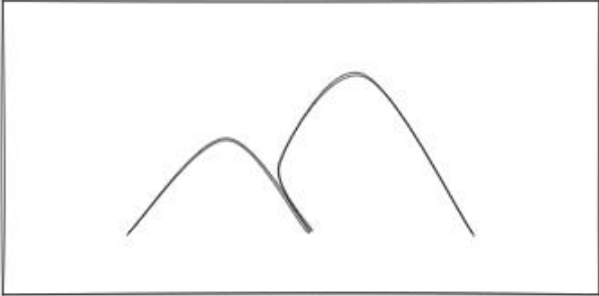
ADD PLANT MODAL

The screenshot shows the 'Add New Plant' modal within the Ithilien app. The app's header includes the 'Ithilien' logo, a 'My Garden' button with a star icon, a 'Calendar' button, and a user profile icon. The modal itself has a title 'Add New Plant' and a close button. It contains a 'Photo' section with a placeholder image of two hills. Below this are two text input fields: 'Nickname' with the placeholder 'Give your plant a name...' and 'Species' with the placeholder 'Search for plant species...'. A small note 'Powered by Perennial API' is located below the 'Species' field. At the bottom of the modal is a button with a left arrow and the text 'Back to My garden'.

The Add Plant modal allows users to add new plants to their garden. Users can upload a photo of their plant, assign a custom nickname, and search for their plant's species. When a species is selected, the app retrieves care information (watering frequency, sunlight requirements, etc.) from the Perennial API. Cancel and Add Plant buttons appear at the bottom to confirm or discard the entry.

 Ithilien

[My Garden](#) [Calendar](#) 



Leafy
Boston Fern (*Nephrolepis exaltata*)

Mark as Watered

Care Requirements

Watering	Every 5-7 days
Sunlight	Indirect light
Humidity	High (50-80%)
Temperature	15-24°C

Powered by Perennia API

Care Status

 Needs Water! - Last watered 9 days ago

Care History

Feb 7	Watered
Jan 31	Watered
Jan 24	Watered
Jan 17	Added to Garden

[Edit Plant](#) [Delete Plant](#)

The Plant Profile page displays detailed information about a single plant. At the top, users can view the plant's photo, nickname, and species name. A prominent "Mark as Watered" button allows users to quickly log that action. The Care Requirements section displays species-specific information retrieved from the Perennia API, including watering frequency, sunlight needs, humidity, and temperature preferences. The Care Status section indicates whether the plant currently needs attention. Below that, the Care History log shows a chronological list of past actions (watering, fertilizing, etc.) with the dates these events occurred. Edit and Delete buttons allow users to modify or remove the plant from their garden.

 Ithilien

My Garden

Calendar



Watering Schedule

← February 2026 →

Sun	Mon	Tue	Wed	Thu	Fri	Sat
1	2	3	4 leafy	5	6	7
8	9 Today Hark	10	11 leafy	12	13	14 Fernando
15	16	17	18 leafy	19	20	21
22	23 Hark	24	25 leafy	26	27	28 Fernando

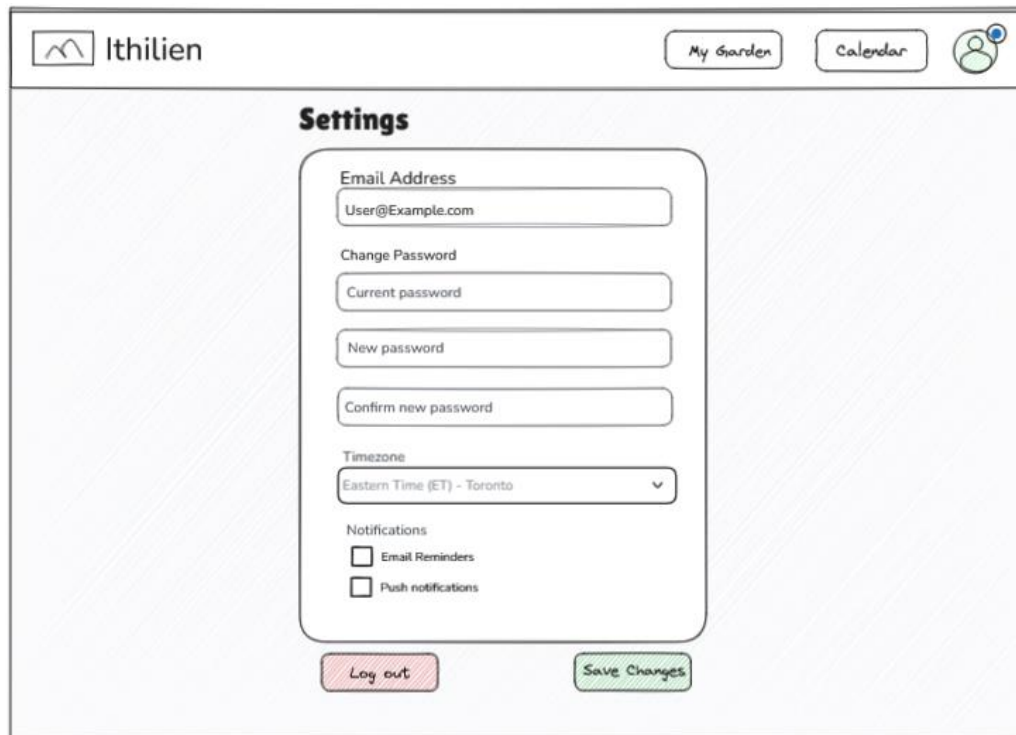
 Scheduled watering

 Completed

 Overdue

The Calendar page provides a monthly overview of all watering tasks. Each task is color-coded by status: blue for upcoming, green for completed, and red for overdue. Users can navigate between months using the arrow buttons and click on individual tasks to mark them as completed directly from this view.

USER SETTINGS



The image shows a web application interface for 'Ithilien'. At the top, there is a header bar with the 'Ithilien' logo on the left and two buttons, 'My Garden' and 'Calendar', on the right. Below the header, the main content area is titled 'Settings'. Inside this area, there is a central form with several sections: 'Email Address' with a text input field containing 'User@example.com'; 'Change Password' with three text input fields labeled 'Current password', 'New password', and 'Confirm new password'; 'Timezone' with a dropdown menu showing 'Eastern Time (ET) - Toronto'; and 'Notifications' with two checkboxes, 'Email Reminders' and 'Push notifications', both of which are currently unchecked. At the bottom of the form, there are two buttons: 'Log out' (pink) and 'Save Changes' (green).

Ithilien

My Garden Calendar

Settings

Email Address

User@example.com

Change Password

Current password

New password

Confirm new password

Timezone

Eastern Time (ET) - Toronto

Notifications

☐ Email Reminders

☐ Push notifications

Log out Save Changes

The User Settings page allows users to manage their account preferences. Users can update their email address, change their password (requiring current password confirmation), and select their time zone for accurate reminder scheduling. Notification preferences let users enable or disable email and push notifications. A logout option and account deletion button are also available at the bottom of the page.

APIS AND INTERFACES

DATA FLOW

1. User searches "monstera" in Add Plant modal

Frontend:

GET /api/species/search?q=monstera

2. Backend checks species_cache table

If not in cache =

1. Call Perennial API
2. Store in cache
3. Return to frontend

3. User selects species, adds nickname like "Monty", uploads photo

Frontend:

POST /api/plants { species_id: 155, nickname: "Monty", photo: [base64], last_watered: "2026-02-10" }

4. Backend:

1. Validates JWT token
2. Saves photo to Azure Blob Storage (or local in dev)
3. Inserts plant record in MySQL
4. Calculates next watering date
5. Returns plant object

5. Frontend displays new plant card in My Garden

EXTERNAL API: PERENUAL.COM

Base URL	https://perenual.com/api
Authentication	API Key required append <code>?key=[YOUR-API-KEY]</code> to all requests
Rate Limits	100 requests/month
Rate Limit Solution	When the user searches a plant, the backend checks the <code>species_cache</code> table in the database first If found: Return cached data with 0 Perenual API calls If not found: 1. Call Perenual API 2. store result 3. return data Popular plants will only require 1 API call total, even with many users Table: <pre>CREATE TABLE species_cache (perenual_id INT PRIMARY KEY, common_name VARCHAR(255), scientific_name TEXT, watering_frequency VARCHAR(50), watering_days_min INT, watering_days_max INT, sunlight TEXT, care_level VARCHAR(50), indoor BOOLEAN, poisonous_to_pets BOOLEAN, description TEXT, image_url VARCHAR(500), cached_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP);</pre>

SPECIES LIST

Command	GET https://perenual.com/api/v2/species-list?key=[API-KEY]&q=monstera
Purpose	Gets a list of matching plant species Used for Add Plant to show list of species to choose from
Parameters	q (string): Search query (e.g., "monstera", "snake plant") indoor=1 (boolean): Filter for indoor plants only (since this is a houseplant app) page (int): Pagination (default: 1, 30 results per page)
Response Fields	id: Perenual species ID (needed for details endpoint) common_name: Display name scientific_name: Latin name default_image.thumbnail: Image for search results
Example Response (json)	<pre>{ "data": [{ "id": 155, "common_name": "Monstera Deliciosa", "scientific_name": ["Monstera deliciosa"], "default_image": { "thumbnail": "https://perenual.com/storage/..." } }] }</pre>

PLANT DETAILS

Command	GET https://perenual.com/api/v2/species/details/[ID]?key=[API-KEY]
Purpose	Gets the details of a chosen plant Used for plant details, care requirements and watering schedule
Parameters	{id}: Perenual species ID from search results
Response Fields	watering: General frequency ("Frequent", "Average", "Minimum") watering_general_benchmark.value: Specific days between waterings (e.g., "5-7") sunlight: Array of light requirements (["full sun", "part shade"]) care_level: Difficulty ("Easy", "Medium", "Hard") indoor: Is it suitable for indoors? description: Care tips and plant info poisonous_to_pets: Safety warning default_image: Plant photo URLs
Example Response (json)	<pre>{ "id": 155, "common_name": "Monstera Deliciosa", "watering": "Average", "watering_general_benchmark": { "value": "7-10", "unit": "days" }, "sunlight": ["bright indirect light"], "care_level": "Easy", "indoor": true, "poisonous_to_pets": true, "description": "Popular tropical houseplant..." }</pre>

INTERNAL API

Base URL	/api
Authentication	Users receive a signed token upon successful login Token holds user ID and email (not password) Token must be included in Authorization header to protect endpoints Backend verifies token signature to prevent tampering Tokens expire after 24 hours for security

ENDPOINTS:

AUTHENTICATION:

Endpoint	Method	Purpose	Authorization Required?
/auth/register	POST	Create New User	No
/auth/login	POST	Login User	No
/auth/logout	POST	Logout User	Yes

USER MANAGEMENT:

Endpoint	Method	Purpose	Authorization Required?
/user/profile	GET	Get user settings	Yes
/user/account	DELETE	Delete user account	Yes
/user/profile	PUT	Update email / password / time zone	Yes

PLANT MANAGEMENT:

Endpoint	Method	Purpose	Authorization Required?
/plants	GET	Get all user's plants by care urgency	Yes
/plants	POST	Add new plant to user list	Yes
/plants/:id	GET	GET info for single plant	Yes
/plants/:id	PUT	Update plant info	Yes
/plants/:id	DELETE	Remove plant from user's list	Yes
/plants/:id/water	POST	Mark plant as watered	Yes
/plants/:id/history	GET	Get care history log for plant	Yes

CALENDAR:

Endpoint	Method	Purpose	Authorization Required?
/calendar	GET	GET monthly watering schedule	Yes
/reminders	GET	GET upcoming reminders set for user	Yes

ERROR HANDLING

- Validate input and authentication requests
- All errors must return a consistent JSON format

- Frontend displays user-friendly messages detailing errors
- Critical errors are logged for monitoring
- Perenual API failures use cached species data as a fallback

ERROR CODES

Error Code	Description
200	Success
400	Invalid Input
401	Not Authenticated
403	Improper Permissions
404	Resource Not Found
500	Server Error
503	External Service Unavailable

ERROR SCENARIOS

Error Category	Error Types
Authentication	- Expired token - Invalid Credentials - Email Already Exists in Database - Password too weak - Unauthorized Access
Validation	- Missing field - Invalid Format
Resources	- Plant not found
External APIs	- Error with Perenual requests
Database	- Connection Error - Data Not Found - Database Error

DEPLOYMENT AND RELEASE STRATEGY

LOCAL DEV ENVIRONMENT

Developers will use a local MySQL server, and a chosen path in their filesystem to perform relational DB and file storage operations.

ENVIRONMENT VARIABLES

For local deployment, a **.env** file will be used to store connection strings, path prefixes, and a flag indicating whether the app is in development or production which will be used by the **Image Storage Adapter**.

While deployed via Azure App Services, these environment variables will be provided via **Azure App Settings**.

Perenual API Exception:

The one notable exception to this approach will be the Perenual API, which has no direct “dev mode” equivalent. All development *and* production requests to this API will go directly to the <https://perenual.com> endpoints.

IMAGE STORAGE ADAPTER

An Image Storage Adapter interface will be defined with two concrete implementations:

- One for storing files to the local filesystem during development
- One for interacting with Azure Blob Storage in deployed environments

Environment Variables will control which implementation is active at runtime.

DEPLOYMENT TO AZURE APP SERVICES

Deployment of the frontend and backend code to the Azure App Services platform will occur through the Microsoft official VS Code extension by manually pushing to production.

No CI/CD pipelines or automation tools will be used.